

EECS 498 • AASE • FALL 2026

From pair-coding to agent orchestration

LECTURE 06 • SEP 17, 2026

TODAY

Three parts

1. The project: `ai-assistant`
2. Working with the model
3. From edit blocks to tool calls

The project

AI-ASSISTANT, DUE OCTOBER 6

Your own pair-programmer

- A terminal chat about files you choose
- The model proposes edits, you approve
- Each approved change becomes a commit

Aider, the tool you have used for three weeks. Now you write one.

What staff supply and what you decide

Staff supply

The contract: 7 features

A starter loop, `bin/assistant`, `bin/test`

`ENDPOINT.md` and `ACCEPTANCE.md`

You decide

Modules, interfaces and state

Estimator, trimming, file delimiter

Error messages and commit ownership

Your tests

The order of work

1. Read the packet, `EXAMPLES.md` first
2. Design and diagrams, reviewed cold
3. Increments, each spec written first
4. Live evidence, usage, reflection

Design is committed before application code.

How the build is graded

Half	Points
Specification and design	50
Implementation and tests	50

- Specs before code, read from commit order
- No hidden tests or requirements
- No points for suffering

Dates and deliverables

When	What
After this lecture	The packet, in your repository
Before any app code	<code>design/SYSTEM.md</code> and diagrams
Thu Sep 24, evening	Hackathon 1, separate
Tue Oct 6, 11:59 PM	The build is due

One repository, all semester. Commit your Aider history.

Working with the model

UNDERSTANDING WHAT IT WRITES AND WHAT IT SAYS

Who understands the code

- The model writes most of the lines
- You own the decisions and outcomes
- Understanding at the right level

What you hold in your head

- The design and why it is that way
- Where you are in the build
- What is actually verified
- What the model only claimed

The model keeps none of this between sessions.

Judgment at the right distance

Distance	What it looks like
Too close	Reading every line it wrote
Too far	Accepting what you never checked
About right	Checking contracts and outcomes

Not every line. Every decision.

A working relationship needs **shared names**.

Reference points

Name	What it points at
F6	Previewing and applying edits
design/SYSTEM.md §6	Where your safety decisions live
specs/03-preview.md	The increment that delivers it
test_stale_preview_refused	What checks it

A short, stable name that you, the model and staff resolve the same way.

Decision codes in a session

model: decisions before I write the slides

Code	Decision
------	----------

D1	The live demo is Claude Code in a repository
----	--

D2	The approval layer moves out of this lecture
----	--

D3	READ and RUN use made-up markers, not F5's
----	--

D4	The tool-call build-up is slides, not live code
----	---

D5	The chat break becomes a discussion question
----	--

... D6 to D18

Answering by code

```
you      D1 yes. D3 yes. D4 no, do READ live. D5 yes.  
  
model    D1, D3, D5 kept.  
          D4 changed: READ is live, RUN stays on slides.  
          D2 still open. Keep it out?  
  
you      D2 yes.
```

Five of eighteen decisions, in two lines.

Names the packet leaves to you

- SPEC names its 7 features
- You name your acceptance criteria
- Each of your names traces to a feature

"Restating SPEC.md section 4 earns nothing." RUBRIC.md

Prompting by name

By paraphrase

fix the approval stuff so
it's safer

By name

specs/03, task 2 only.
Deliver F6-3.
Make test_stale_preview_refused
pass. Edit preview.py only.

Asking for answers in your names

Before I review: which criteria does this change deliver,
what did you run, and what did you not verify?

Delivers	F6-3
Touched	assistant/preview.py
Ran	nothing
Unverified	F6-3 against a real target repository

Project state in one line

```
specs/03 · F6-3 done · F6-4 red: test_write_failure_restores
```

- A message to staff or a neighbor
- The first line of a fresh session
- A commit message

WHERE IT BREAKS

How names fail

- A paraphrase replaces the ID
- The model invents an ID
- The named file is not in context
- A name changes meaning mid-build

LIVE DEMO

Claude Code in a repository

- Name the task by reference point
- Ask for the report in those names
- Check each claim

Other ways to read a model's output

Reference points are one construct.

Name **one other habit or construct** that makes a model's output easier to understand, or otherwise facilitates communication.

One line each. We will collect five.

From edit blocks to tool calls

GROWING THE ONE COMMAND YOUR PROGRAM HAS

The one command your program has

```
greet.py
<<<<<< SEARCH
print("Hello")
=====
print("Hi")
>>>>>> REPLACE
```

Text from the model. Your code parses it and acts, on yes.

When the model needs a file

```
you      why does greet() print twice?
model    I can't tell without main.py, which calls greet().
you      /add main.py
you      ok, look again
```

The model knew what it needed. Only you could fetch it.

THE QUESTION

Why not let it read

- It already names the file it needs
- Reading changes nothing on disk
- You were only relaying

So give it a way to ask.

ADD A COMMAND

A READ command

I need to see the tests before I change anything.

```
<<<<<<< READ  
tests/test_cli.py  
>>>>>>> END
```

Your code reads the file and sends its contents back.

ADD ANOTHER

A RUN command

Let me check the suite first.

```
<<<<<< RUN
```

```
bin/test
```

```
>>>>>> END
```

Your code runs it and sends the output back.

A dispatcher for three commands

```
for block in parse_blocks(reply):
    if block.kind == "EDIT":
        result = propose_edit(block.path, block.search, block.replace)
    elif block.kind == "READ":
        result = read_file(block.path)
    elif block.kind == "RUN":
        result = run_command(block.command)
    messages.append({"role": "user", "content": result})
```

One shape for every command

Block	Name	Arguments
SEARCH/REPLACE	edit_file	path, search, replace
READ	read_file	path
RUN	run_command	command

A name plus arguments. That is a **tool call**.

The shape of a tool call

```
{"tool_calls": [  
  {"name": "read_file", "arguments": {"path": "tests/test_cli.py"}},  
  {"name": "run_command", "arguments": {"command": "bin/test"}},  
  {"name": "edit_file", "arguments": {"path": "greet.py", "search": "...", "replace": "..."}}  
]}
```

Same three commands, parseable without a custom format.

The model never runs anything. It
asks.

Who sends the result back

```
you      why does bin/test fail?
model    READ tests/test_cli.py
you      (contents) keep going
model    RUN bin/test
you      (output) keep going
model    SEARCH/REPLACE tests/test_cli.py
```

Every "keep going" is you, being the loop.

The loop that picks its step

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text          # the model is done
    for call in reply.tool_calls:
        result = execute(call)     # your code, your rules
        messages.append(result)
```

Before and after the loop

- The model still only writes text
- Your code still does every action
- The model now picks the next step

Only the last line changed.

What the loop still needs

- A cap on iterations and tokens
- A detector for repeated calls
- A clean interrupt, mid-loop
- Approval before `run_command`

Undo covers files. It does not cover a shell.

This week and the next two

When	What
Tonight	Read <code>EXAMPLES.md</code> , then SPEC
Before any app code	Design, diagrams, cold review
Thu Sep 24, evening	Hackathon 1
Tue Oct 6, 11:59 PM	The build is due

Questions?